

SQLite3 in Python — Notes

Using the built-in sqlite3 module: connecting, creating cursors, inserting, updating, deleting, and querying data.

1. Connecting to a Database

```
conn = sqlite3.connect("student.db")
```

- Creates the database file if it doesn't already exist, otherwise connects to it.
- **conn** is the connection object — it manages the actual link to the database file.

2. Cursor Object

```
cursor = conn.cursor()
```

- The cursor is what you use to actually run SQL commands and fetch results.
- Think of **conn** as the connection to the house, and **cursor** as the hand that does the work inside.

3. Sample Data (Python list of tuples)

```
students = [  
    ("Ram", 22, "Btech"),  
    ("Raman", 20, "BCA"),  
    ("Mohit", 23, "BBA")  
]
```

- A list of tuples — each tuple represents one row to insert.

4. Reading Data — SELECT

```
cursor.execute("SELECT * FROM students")  
data = cursor.fetchall()  
for row in data:  
    print(row)
```

- **execute()** runs a single SQL query.
- **fetchall()** returns all matching rows as a list of tuples.
- Other options: **fetchone()** (one row), **fetchmany(n)** (n rows).

5. Inserting Multiple Rows — executemany

```
cursor.executemany("""
    INSERT INTO students (name, age, course) VALUES (?, ?, ?)
""", students)
conn.commit()
```

- **executemany()** runs the same query repeatedly, once per tuple in the list — efficient for bulk inserts.
- **?** are placeholders — always use these instead of string formatting (f-strings) to avoid SQL injection.
- **conn.commit()** saves changes permanently. Without **commit()**, insert/update/delete changes are not written to disk.

6. Updating Data — UPDATE

```
cursor.execute("""
    UPDATE students SET course=? WHERE id=?
""", ("MBA", 11))
conn.commit()
```

- Updates the course column to "MBA" for the row where id = 11.
- Always pair UPDATE with a WHERE clause — without it, every row gets updated.

7. Note on Tuples

```
a = (10,)
print(type(a)) # <class 'tuple'>
```

- A single-element tuple needs a trailing comma — (10) is just an integer in parentheses, not a tuple.
- This matters because **execute()** expects a tuple of parameters, even for one value: ("MBA", 11) or (5,).

8. Deleting Data — DELETE

```
cursor.execute("""
    DELETE FROM students WHERE id=?
""", (5,))
conn.commit()
```

- Deletes the row where id = 5.
- Again, WHERE is essential — DELETE FROM students with no WHERE clause deletes all rows.

Quick Reference Table

Operation	Method	Needs commit()?
Create/connect DB	sqlite3.connect()	No
Run one query	cursor.execute()	Only for INSERT/UPDATE/DELETE
Run many inserts	cursor.executemany()	Yes
Read all rows	cursor.fetchall()	No
Save changes	conn.commit()	—
Close connection	conn.close()	(good practice)

Key Takeaways

- execute vs executemany: single query vs. same query run for multiple parameter sets.
- commit() is required after any change (INSERT/UPDATE/DELETE) — reads (SELECT) don't need it.
- Always use ? placeholders, never insert Python variables directly into SQL strings.
- WHERE clause is critical for UPDATE/DELETE to avoid affecting the whole table.
- Don't forget conn.close() at the end of your script to release the database connection properly.